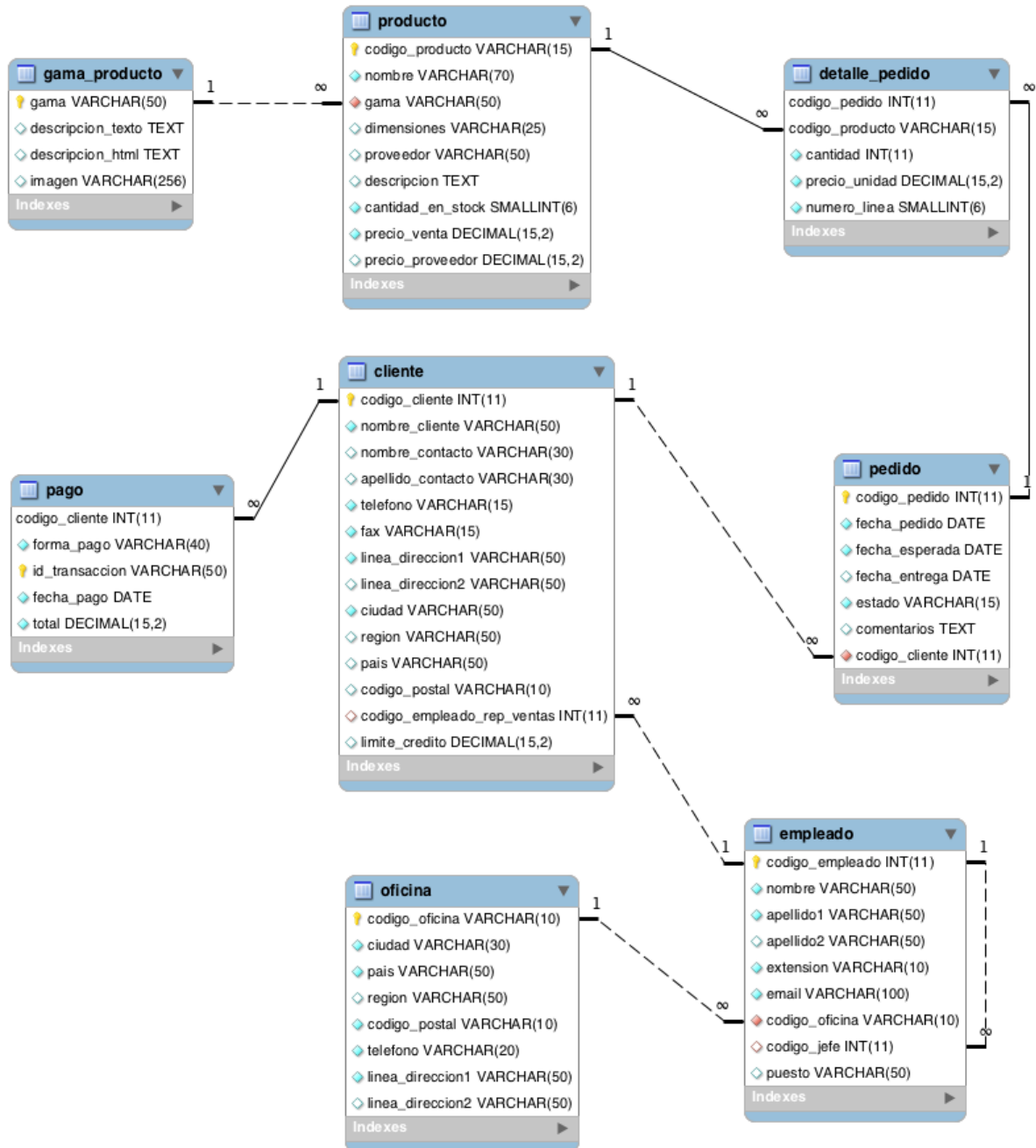


Consultas Group By, Having y Funciones de Agregación



1. ¿Cuántos empleados hay en la compañía?
`select count(*) from empleado;`
2. ¿Cuántos clientes tiene cada país?
`select pais, count(*) from cliente group by pais;`
3. ¿Cuál fue el pago medio en 2009?
`select avg(total) from pago where year(fecha_pago) = 2009;`

4. ¿Cuántos pedidos hay en cada estado? Ordena el resultado de forma descendente por el número de pedidos.
`select estado, count(*) from pedido group by estado order by count(*) desc;`
5. Calcula el precio de venta del producto más caro y más barato en una misma consulta.
`select max(precio_venta), min(precio_venta) from producto;`
6. Calcula el número de clientes que tiene la empresa.
`select count(*) from cliente;`
7. ¿Cuántos clientes tiene la ciudad de Madrid?
`select count(*) from cliente where ciudad = 'Madrid';`
8. ¿Calcula cuántos clientes tiene cada una de las ciudades que empiezan por M?
`select ciudad, count(*) from cliente where ciudad like 'M%' group by ciudad;`
9. Devuelve el nombre de los representantes de ventas y el número de clientes al que atiende cada uno.
`select r.nombre, count(*) from cliente c join empleado r on c.codigo_empleado_rep_ventas = r.codigo_empleado group by r.codigo_empleado;`
10. Calcula el número de clientes que no tiene asignado representante de ventas.
`select count(*) from cliente where codigo_empleado_rep_ventas is null;`
11. Calcula la fecha del primer y último pago realizado por cada uno de los clientes. El listado deberá mostrar el nombre y los apellidos de cada cliente.
`select c.nombre_contacto, c.apellido_contacto, min(p.fecha_pago), max(p.fecha_pago) from pago p join cliente c on p.codigo_cliente = c.codigo_cliente group by c.codigo_cliente;`
12. Calcula el número de productos diferentes que hay en cada uno de los pedidos.
`select codigo_pedido, count(*) from detalle_pedido group by codigo_pedido;`
13. Calcula la suma de la cantidad total de todos los productos que aparecen en cada uno de los pedidos.
`select codigo_pedido, sum(cantidad*precio_unidad) from detalle_pedido group by codigo_pedido;`
14. Devuelve un listado de los 20 productos más vendidos y el número total de unidades que se han vendido de cada uno. El listado deberá estar ordenado por el número total de unidades vendidas.
`select codigo_producto, sum(cantidad) from detalle_pedido group by codigo_producto order by 2 desc limit 20;`
15. La facturación que ha tenido la empresa en toda la historia, indicando la base imponible, el IVA y el total facturado. La base imponible se calcula sumando el coste del producto por el número de unidades vendidas de la tabla detalle_pedido. El IVA es el 21 % de la base imponible, y el total la suma de los dos campos anteriores.
`select sum(cantidad*precio_unidad) as Base_Imponible, 0.21*sum(cantidad*precio_unidad) as IVA, 1.21*sum(cantidad*precio_unidad) as TOTAL_FACTURADO from detalle_pedido;`
16. La misma información que en la pregunta anterior, pero agrupada por código de producto.
`select codigo_producto, sum(cantidad*precio_unidad) as Base_Imponible, 0.21*sum(cantidad*precio_unidad) as IVA, 1.21*sum(cantidad*precio_unidad) as TOTAL_FACTURADO from detalle_pedido group by codigo_producto;`

17. La misma información que en la pregunta anterior, pero agrupada por código de producto filtrada por los códigos que empiecen por OR.

```
select codigo_producto, sum(cantidad*precio_unidad) as Base_Imponible,  
0.21*sum(cantidad*precio_unidad) as IVA, 1.21*sum(cantidad*precio_unidad) as  
TOTAL_FACTURADO from detalle_pedido where codigo_producto like 'OR%'  
group by codigo_producto;
```

18. Lista las ventas totales de los productos que hayan facturado más de 3000 euros. Se mostrará el nombre, unidades vendidas, total facturado y total facturado con impuestos (21% IVA).

```
select p.nombre, sum(cantidad) as Unidades_Vendidas,  
sum(d.cantidad*d.precio_unidad) as Total_Facturado,  
1.21*sum(d.cantidad*d.precio_unidad) as TOTAL_FACTURADO_IVA from  
detalle_pedido d join producto p on d.codigo_producto = p.codigo_producto group by  
d.codigo_producto having sum(d.cantidad*d.precio_unidad)>3000;
```